

REDES DE COMPUTADORES

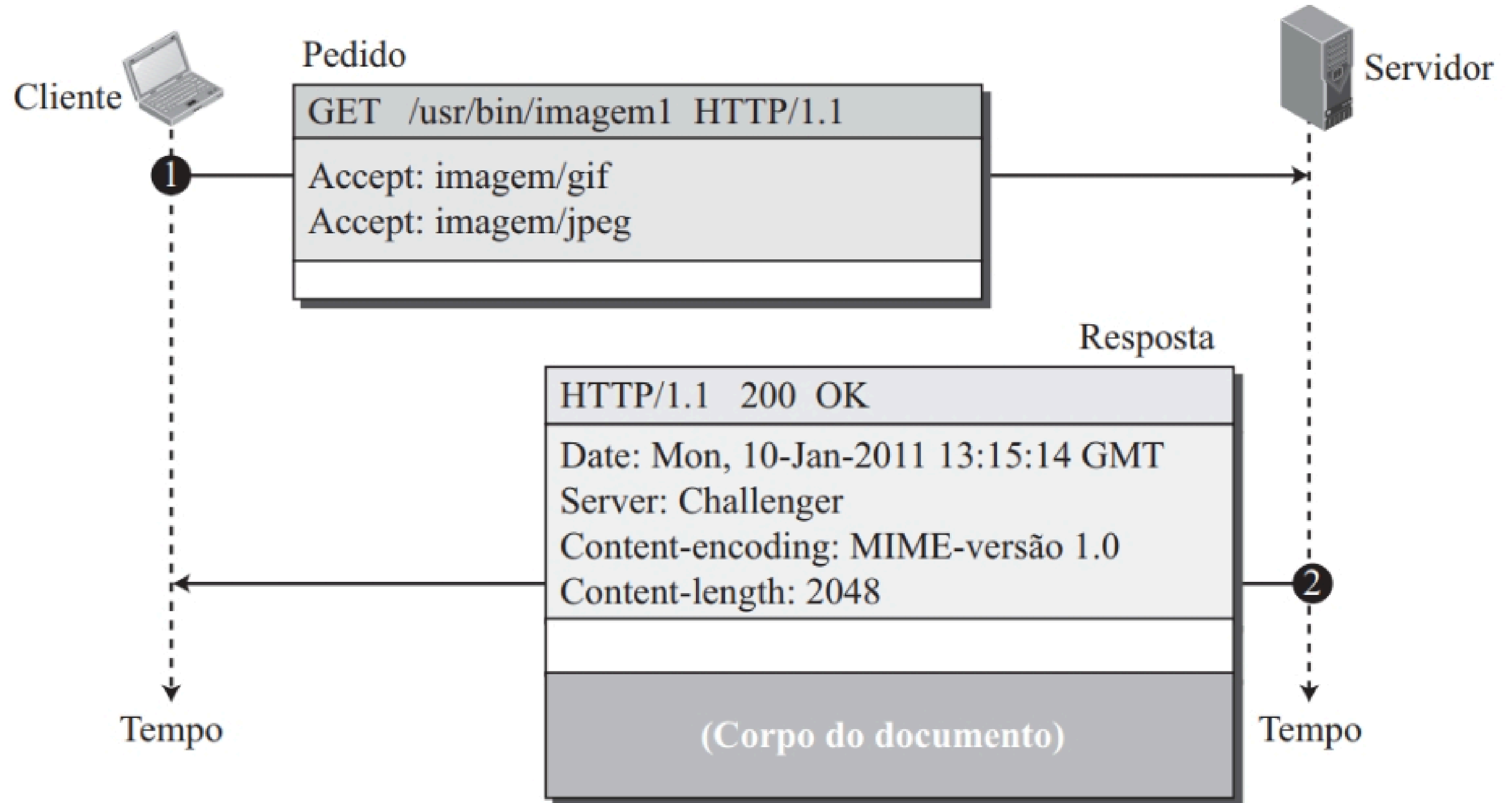
Aula 05 - Camada de Aplicação - Parte III

- HTTP - GET e POST
- HTTP - Cookies
- HTTP - Proxy

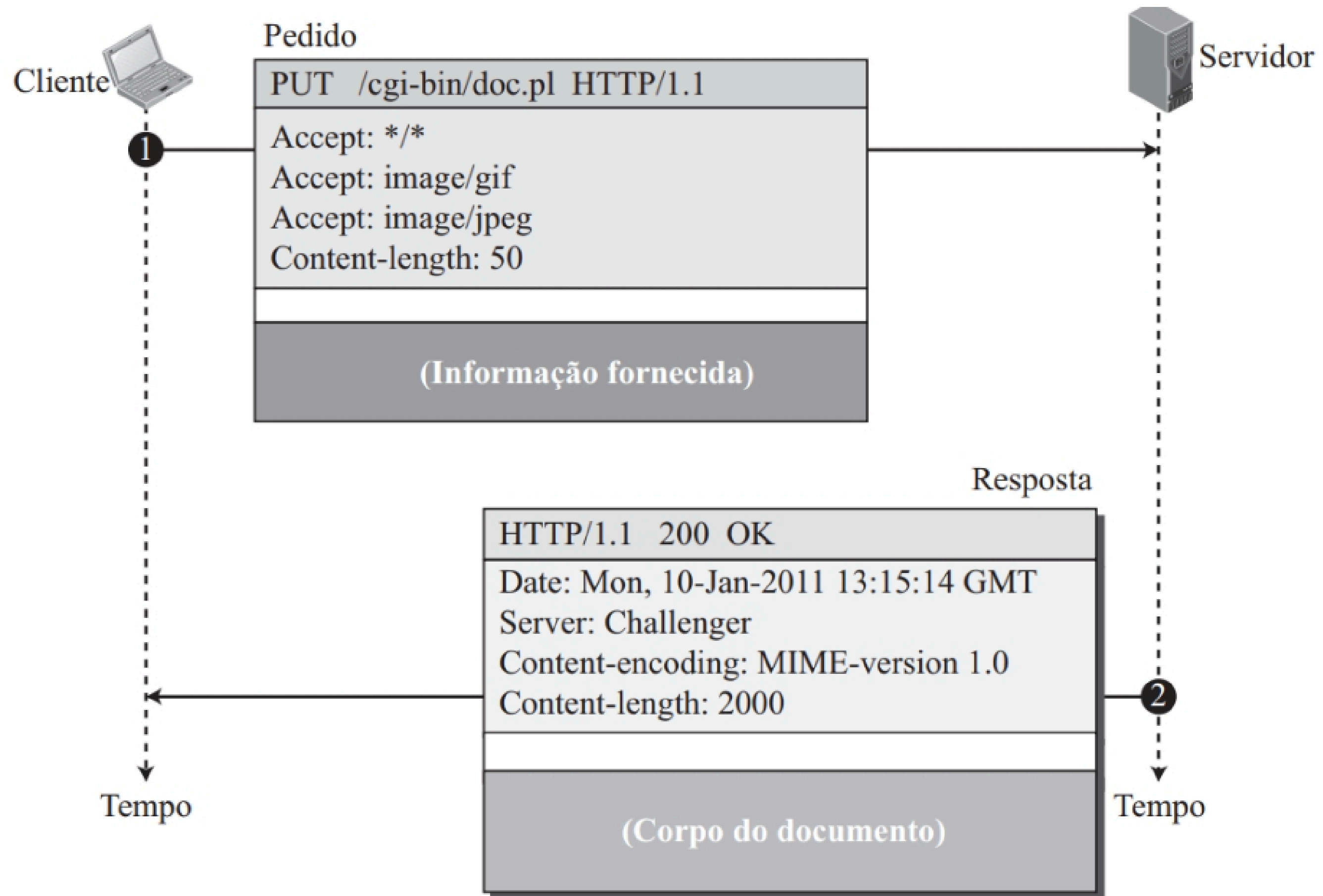


Lorena de S. B. Borges

HTTP - MÉTODO GET



HTTP - MÉTODO PUT





HTTP - PEDIDO CONDICIONAL

- UM CLIENTE PODE ADICIONAR UMA CONDIÇÃO EM SEU PEDIDO.
- O SERVIDOR ENVIARÁ A PÁGINA WEB SOLICITADA SE A CONDIÇÃO FOR SATISFEITA, OU INFORMARÁ O CLIENTE CASO CONTRÁRIO
- UMA DAS CONDIÇÕES MAIS COMUNS IMPOSTAS PELO CLIENTE REFERE-SE À DATA E HORA EM QUE A PÁGINA WEB FOI MODIFICADA
- O CLIENTE PODE ENVIAR A LINHA DE CABEÇALHO **IF-MODIFIED-SINCE** PARA NOTIFICAR AO SERVIDOR QUE PRECISA DA PÁGINA APENAS SE ELA FOI MODIFICADA APÓS UM CERTO PONTO

HTTP - PEDIDO CONDICIONAL

O exemplo a seguir mostra como um cliente pode impor a condição de data e hora de modificação em um pedido.

```
GET http://www.commonServer.com/information/arquivo1 HTTP/1.1
```

Linha de solicitação

```
If-Modified-Since: Thu, Sept 04 00:00:00 GMT
```

Linha de cabeçalho

Linha em branco

A linha de estado na resposta mostra que o arquivo não foi modificado após o instante de tempo definido. O corpo da mensagem de resposta também se encontra vazio.

```
HTTP/1.1 304 Not Modified
```

Linha de estado

```
Date: Sat, Sept 06 08 16:22:46 GMT
```

Primeira linha de cabeçalho

```
Server: commonServer.com
```

Segunda linha de cabeçalho

Linha em branco

```
(Corpo vazio)
```

Corpo vazio



HTTP - COOKIES

- A WEB ATUAL TEM FUNCIONALIDADES QUE PRECISAM SE LEMBRAR DE ALGUMAS INFORMAÇÕES SOBRE OS CLIENTES, MECANISMO DE **COOKIES** :
 - SITES ESTÃO SENDO USADOS COMO LOJAS VIRTUAIS, PERMITINDO QUE OS USUÁRIOS NAVEGUEM PELA LOJA, SELECIONEM OS ITENS DESEJADOS, COLOQUE-OS EM UM CARRINHO DE COMPRAS VIRTUAL E PAGUEM NO FINAL COM UM CARTÃO DE CRÉDITO.
 - ALGUNS SITES PRECISAM PERMITIR O ACESSO APENAS A CLIENTES REGISTRADOS.
 - ALGUNS SITES SÃO USADOS COMO PORTAIS: O USUÁRIO SELECIONA AS PÁGINAS WEB QUE ELE DESEJA VER.
 - ALGUNS SITES SÃO APENAS AGÊNCIAS DE PUBLICIDADE.



HTTP - COOKIES

- **CRIANDO E ARMAZENANDO COOKIES:**

1. QUANDO UM SERVIDOR RECEBE UM PEDIDO DE UM CLIENTE, ELE ARMAZENA INFORMAÇÕES SOBRE O CLIENTE EM UM ARQUIVO OU EM UMA CADEIA DE CARACTERES:
NOME DE DOMÍNIO DO CLIENTE, O CONTEÚDO DO COOKIE (INFORMAÇÕES QUE O SERVIDOR REUNIU SOBRE O CLIENTE, COMO NOME, NÚMERO DA CARTEIRA DE IDENTIDADE ETC.), UM TIMESTAMP (REGISTRO DE HORÁRIO), E OUTRAS INFORMAÇÕES, DEPENDENDO DA IMPLEMENTAÇÃO.
2. O SERVIDOR INCLUI O COOKIE NA RESPOSTA QUE ENVIA AO CLIENTE.
3. QUANDO O CLIENTE RECEBE A RESPOSTA, SEU NAVEGADOR ARMAZENA O COOKIE NO DIRETÓRIO DE COOKIES.

Maintaining user/server state: cookies

Web sites and client browser use *cookies* to maintain some state between transactions

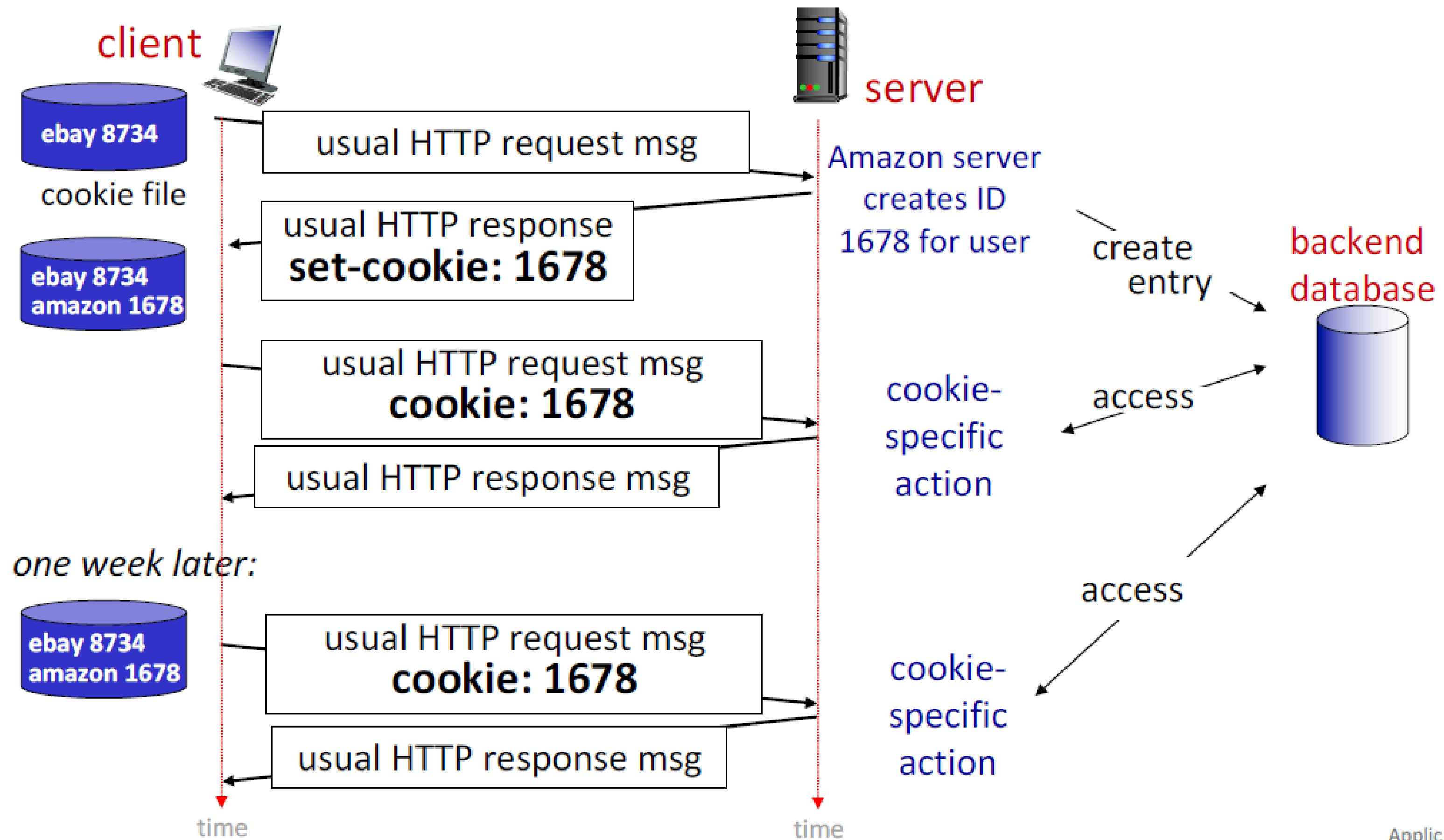
four components:

- 1) cookie header line of HTTP *response* message
- 2) cookie header line in next HTTP *request* message
- 3) cookie file kept on user's host, managed by user's browser
- 4) back-end database at Web site

Example:

- Susan uses browser on laptop, visits specific e-commerce site for first time
- when initial HTTP requests arrives at site, site creates:
 - unique ID (aka “cookie”)
 - entry in backend database for ID
- subsequent HTTP requests from Susan to this site will contain cookie ID value, allowing site to “identify” Susan

Maintaining user/server state: cookies





HTTP - PROXIES

ARMAZENAMENTO TEMPORÁRIO NA WEB: SERVIDOR PROXY

- UM SERVIDOR PROXY É UM COMPUTADOR QUE MANTÉM CÓPIAS DE RESPOSTAS A PEDIDOS RECENTES;
- O CLIENTE ENVIA UMA REQUISIÇÃO HTTP AO SERVIDOR PROXY, QUE VERIFICA SEU CACHE (MEMÓRIA DE ARMAZENAMENTO TEMPORÁRIO);
- SE A RESPOSTA NÃO ESTIVER ARMAZENADA NO CACHE, O SERVIDOR PROXY ENVIA A SOLICITAÇÃO PARA O SERVIDOR CORRESPONDENTE;
- RESPOSTAS RECEBIDAS SÃO ENVIADAS PARA O SERVIDOR PROXY E ARMAZENADAS PARA FUTURAS SOLICITAÇÕES DE OUTROS CLIENTES.



HTTP - PROXIES

BENEFÍCIOS

- O SERVIDOR PROXY REDUZ A CARGA NO SERVIDOR ORIGINAL;
- DIMINUI O TRÁFEGO E MELHORA A LATÊNCIA;
- O CLIENTE DEVE SER CONFIGURADO PARA ACESSAR O PROXY E NÃO O SERVIDOR DE DESTINO;
- O SERVIDOR PROXY FUNCIONA COMO SERVIDOR E CLIENTE.

LOCALIZAÇÃO

- LOCALIZADOS NO LADO DO CLIENTE.
- UM COMPUTADOR CLIENTE PODE TAMBÉM SER USADO COMO UM SERVIDOR PROXY, COM PEQUENA CAPACIDADE;
- EM UMA EMPRESA, UM SERVIDOR PROXY PODE SER INSTALADO NA LAN PARA REDUZIR SEU TRÁFEGO DE ENTRADA E SAÍDA;
- UM ISP COM MUITOS CLIENTES PODE INSTALAR UM SERVIDOR PROXY PARA REDUZIR O TRÁFEGO DE ENTRADA E SAÍDA NA REDE.

HTTP - PROXIES

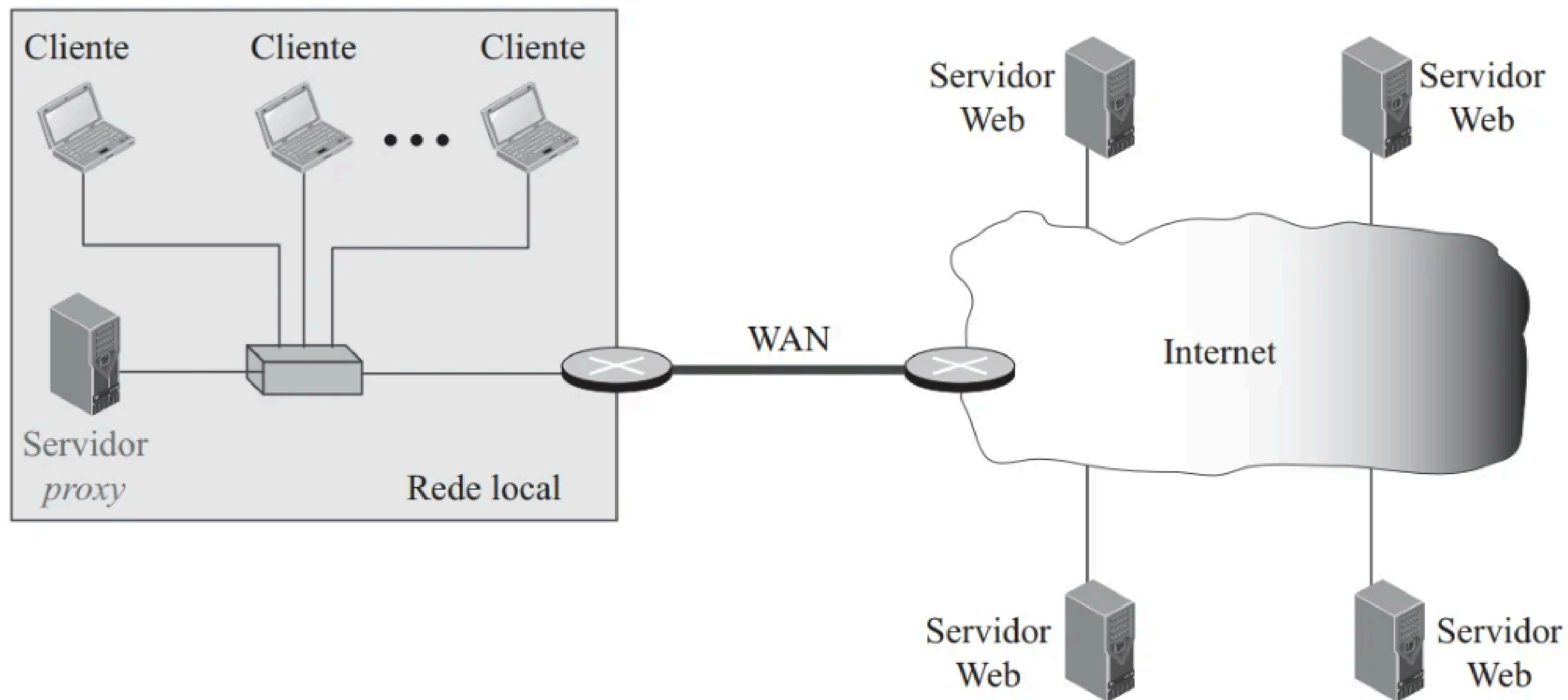
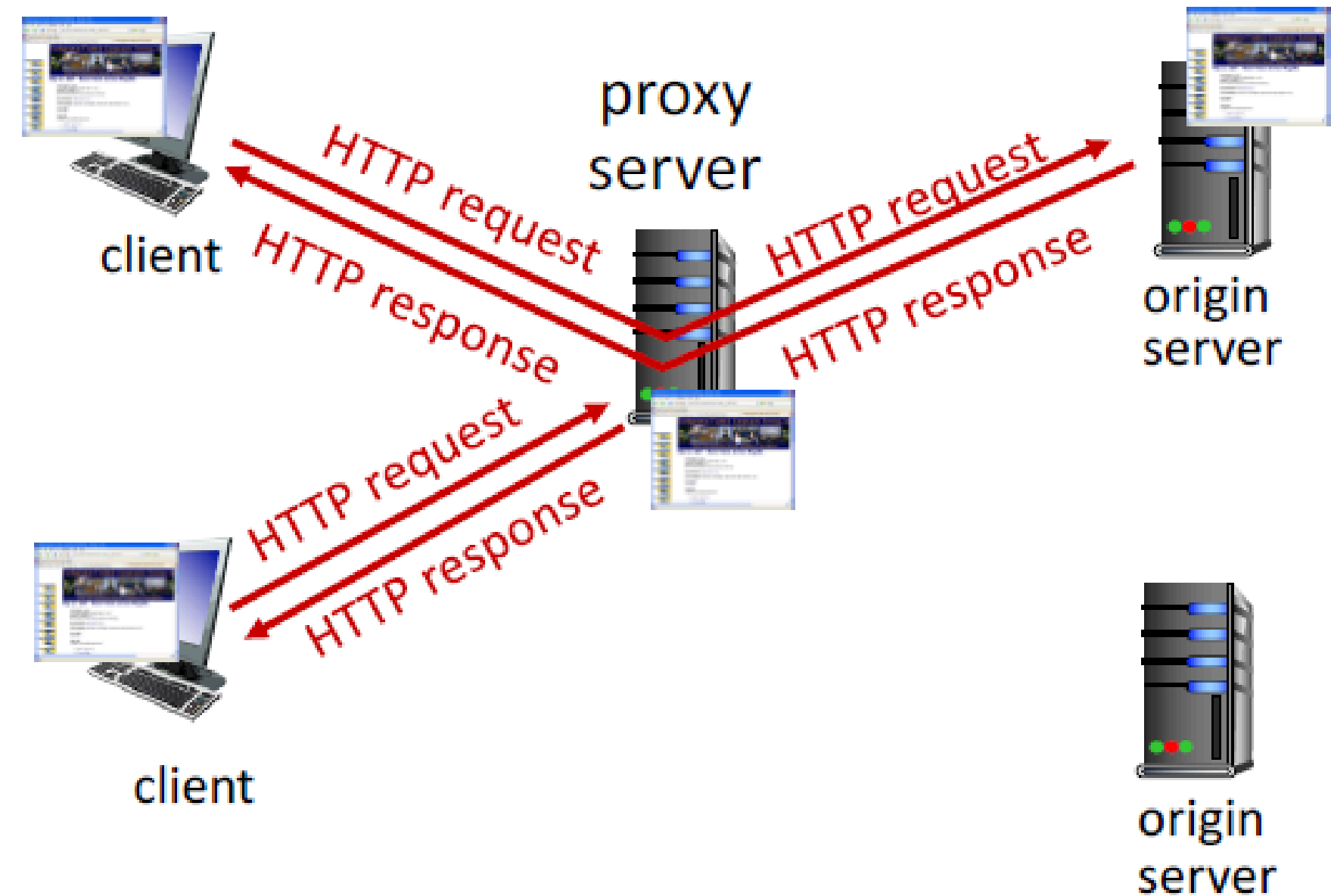


Figura 2.16 Exemplo de um servidor *proxy*.

Web caches (proxy servers)

Goal: satisfy client request without involving origin server

- user configures browser to point to a *Web cache*
- browser sends all HTTP requests to cache
 - *if* object in cache: cache returns object to client
 - *else* cache requests object from origin server, caches received object, then returns object to client



Conditional GET

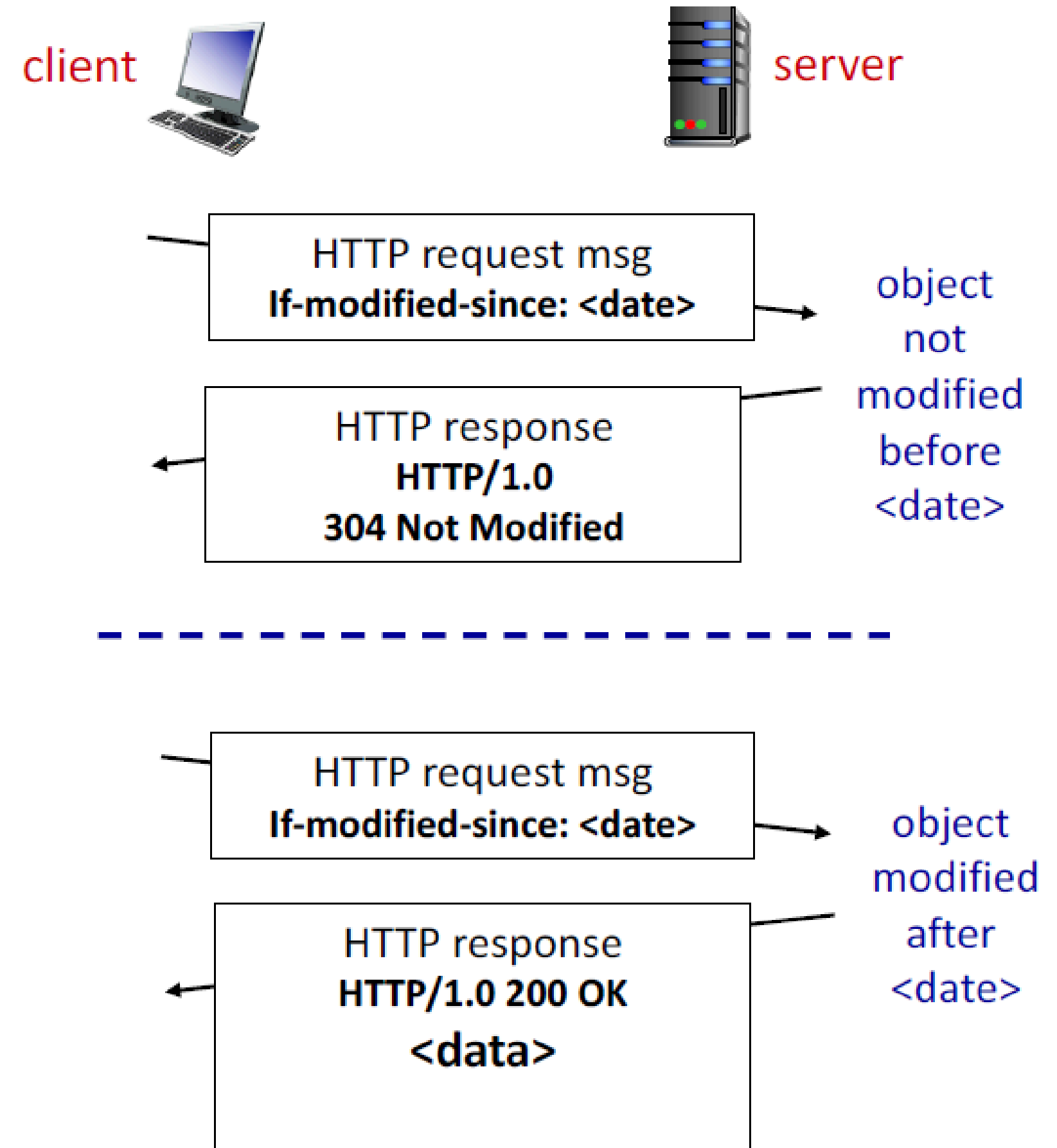
Goal: don't send object if cache has up-to-date cached version

- no object transmission delay
- lower link utilization

■ *cache:* specify date of cached copy in HTTP request

If-modified-since: <date>

■ *server:* response contains no object if cached copy is up-to-date:
HTTP/1.0 304 Not Modified



HTTP/2

Key goal: decreased delay in multi-object HTTP requests

HTTP1.1: introduced multiple, pipelined GETs over single TCP connection

- server responds *in-order* (FCFS: first-come-first-served scheduling) to GET requests
- with FCFS, small object may have to wait for transmission (**head-of-line (HOL) blocking**) behind large object(s)
- loss recovery (retransmitting lost TCP segments) stalls object transmission

HTTP/2

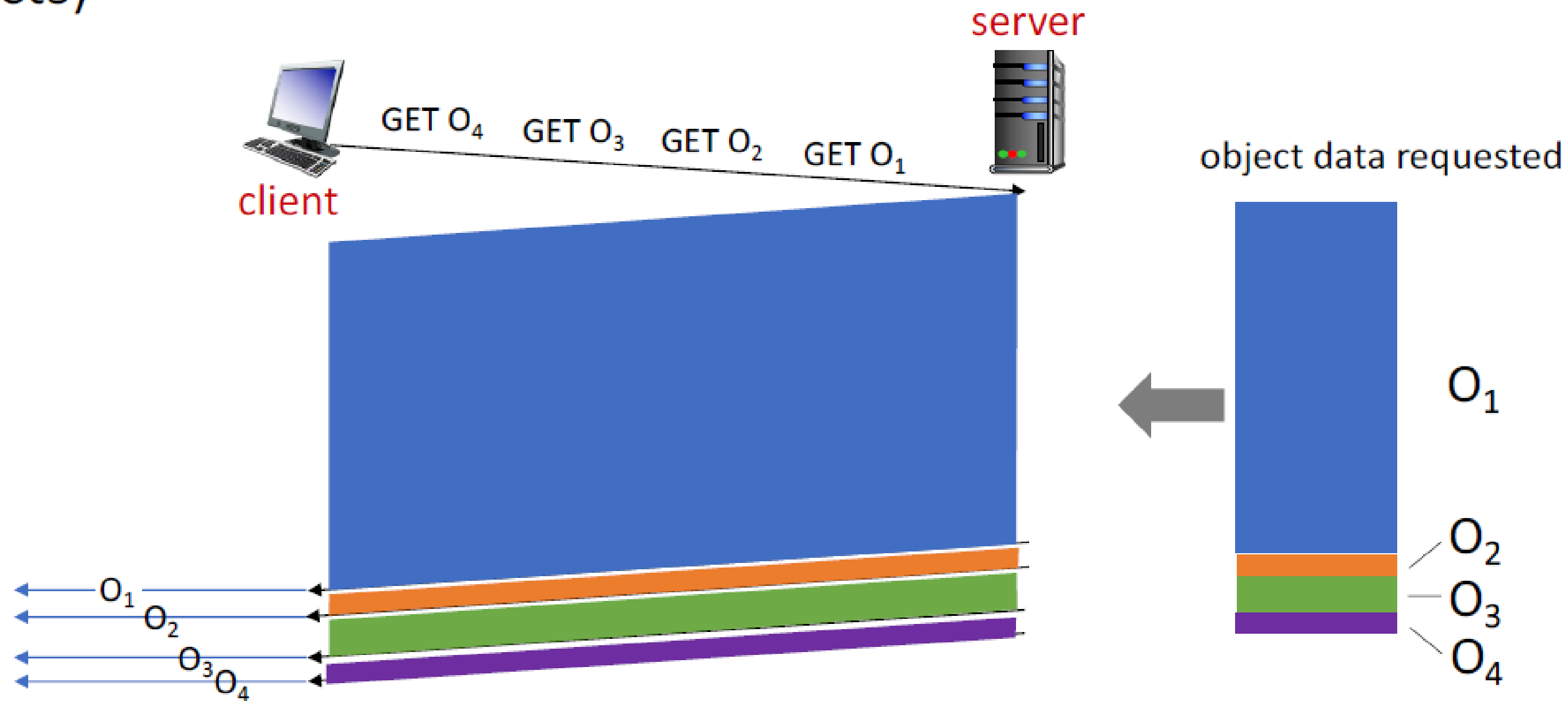
Key goal: decreased delay in multi-object HTTP requests

HTTP/2: [RFC 7540, 2015] increased flexibility at *server* in sending objects to client:

- methods, status codes, most header fields unchanged from HTTP 1.1
- transmission order of requested objects based on client-specified object priority (not necessarily FCFS)
- *push* unrequested objects to client
- divide objects into frames, schedule frames to mitigate HOL blocking

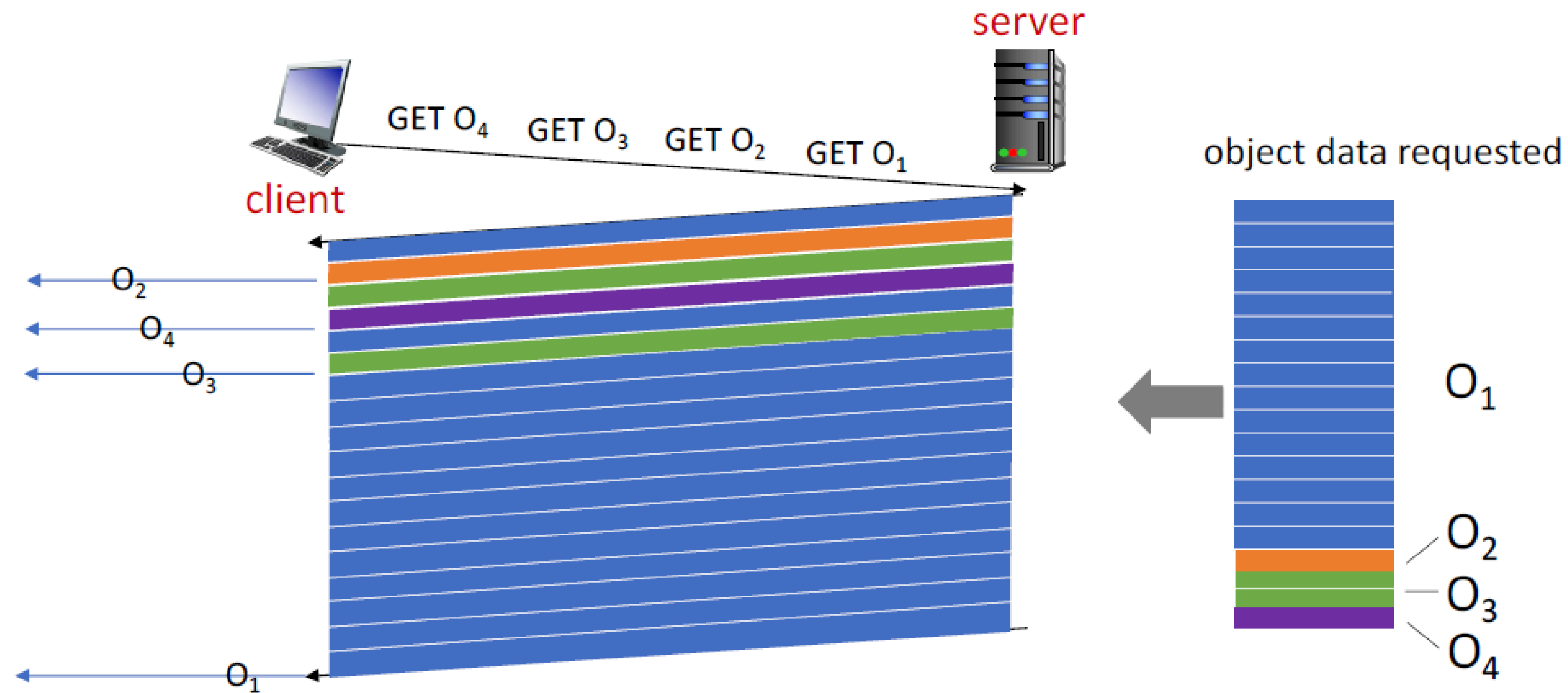
HTTP/2: mitigating HOL blocking

HTTP 1.1: client requests 1 large object (e.g., video file, and 3 smaller objects)



HTTP/2: mitigating HOL blocking

HTTP/2: objects divided into frames, frame transmission interleaved



O₂, O₃, O₄ delivered quickly, O₁ slightly delayed

HTTP/2 to HTTP/3

Key goal: decreased delay in multi-object HTTP requests

HTTP/2 over single TCP connection means:

- recovery from packet loss still stalls all object transmissions
 - as in HTTP 1.1, browsers have incentive to open multiple parallel TCP connections to reduce stalling, increase overall throughput
- no security over vanilla TCP connection
- **HTTP/3:** adds security , per object error- and congestion-control (more pipelining) over UDP
 - more on HTTP/3 in transport layer



REFERÊNCIAS:

- [1] Capítulo 2 - FOROUZAN, B. A. Redes de Computadores: Uma Abordagem Top-Down. AMGH, 2013.
- [2] Capítulo 2 - Computer Networking: A Top-Down Approach 8th edition Jim Kurose, Keith Ross Pearson, 2020.

REDES DE COMPUTADORES

Aula 05 - Camada de Aplicação - Parte III

- HTTP - GET e POST
- HTTP - Cookies
- HTTP - Proxy



Lorena de S. B. Borges